Guide Complet — G05 : Comprendre les Résultats

---

1. Contexte & Problématique (P01)

Qu'est-ce qu'on cherche à répondre ?

La question centrale est : **quel optimiseur permet le meilleur fine-tuning de DistilBERT pour classifier des articles de presse ?**

Ce n'est pas une question triviale. L'optimiseur est l'algorithme qui décide **comment modifier les poids du modèle** à chaque étape d'entraînement. Un mauvais choix peut donner un modèle qui n'apprend pas, qui apprend trop vite et se casse, ou qui mémorise sans généraliser.

Pourquoi c'est important ?

Quand tu fine-tunes un Transformer, tu travailles dans un espace de paramètres **gigantesque** (66 millions de dimensions pour DistilBERT). La surface de perte (loss landscape) dans cet espace est extrêmement complexe, avec des millions de vallées, plateaux, et crêtes. L'optimiseur est la boussole qui te guide dans cet espace.

---

2. Les 3 Optimiseurs en Détail

AdamW (Loshchilov & Hutter, 2019)

Idée de base : Adam maintient pour chaque paramètre une estimation de la moyenne du gradient (direction) et de la variance du gradient (volatilité). AdamW corrige un bug de l'Adam original concernant la régularisation.

Ce qu'il fait concrètement :

Pour chaque paramètre w :

m = moyenne mobile des gradients passés

v = variance mobile des gradients² passés

$w = w - lr \times (m / \sqrt{v}) - lr \times \lambda \times w$

↑ ↑

mise à jour weight decay

adaptative (régularisation)

Intuition visuelle : Imagine que tu descends une montagne dans le brouillard. AdamW se souvient de la direction moyenne dans laquelle tu as marché ces dernières étapes (momentum) ET de comment le terrain a varié (variance). Si le terrain est très irrégulier dans une direction, il ralentit dans cette direction. S'il est régulier, il accélère.

Avantages :

- Très robuste, peu sensible au choix du LR
- Standard de facto pour les Transformers

- Converge généralement vite

Inconvénients :

- Peut trouver des minima "pointus" (sharp minima) qui généralisent moins bien
- Consomme 2× plus de mémoire que SGD (stocke m et v pour chaque paramètre)

SGD + Nesterov Momentum (Sutskever et al., 2013)

Idée de base : La descente de gradient la plus classique, avec un "élan" (momentum) qui accumule la vitesse dans les directions persistantes.

Ce qu'il fait concrètement :

Pour chaque paramètre w :

$v = \beta \times v + \text{gradient}$ (accumulation de vitesse)

$w = w - lr \times v$ (mise à jour)

Nesterov : calcule le gradient UN PAS en avance

→ "regarde avant de sauter"

Intuition visuelle : C'est une bille qui roule sur la surface de perte. Le momentum lui donne de l'élan — elle traverse les petits bosses mais ralentit dans les grandes montées. Nesterov lui permet de "regarder en avant" pour corriger sa trajectoire avant d'arriver à un obstacle.

Avantages :

- Tend à trouver des **minima plus plats** (meilleure généralisation — Keskar et al., 2017)
- Simple, mémoire réduite
- Bien compris théoriquement

Inconvénients :

- Très sensible au LR (trop haut = diverge, trop bas = très lent)
- Difficile à tuner pour les Transformers
- Peut ne pas converger du tout si le LR n'est pas bien choisi

---

Adafactor (Shazeer & Stern, 2018)

Idée de base : Comme Adam mais avec une astuce mathématique qui réduit drastiquement la mémoire en ne stockant pas la matrice de variance complète, seulement ses facteurs (d'où le nom).

Ce qu'il fait concrètement :

Au lieu de stocker v (matrice N×M) :

→ Stocke seulement r (N valeurs) et c (M valeurs)

→ $v \approx r \times c^T$ (approximation rang 1)

Économie mémoire : N×M → N+M

Exemple : matrice 768×768 → 589,824 → 1,536 valeurs

Intuition visuelle : Au lieu de se souvenir de l'historique de chaque case d'un tableau, il se souvient seulement des moyennes par ligne et par colonne. C'est une approximation, mais elle est suffisamment bonne en pratique.

Avantages :

- Mémoire réduite ~4× vs AdamW
- Idéal pour CPU avec RAM limitée ou très grands modèles
- Bon sur les tâches NLP

Inconvénients :

- L'approximation peut légèrement réduire les performances
- Moins testé sur les petits modèles comme DistilBERT

---

3. Concepts Clés pour Interpréter les Résultats

Loss (Perte)

C'est la mesure de l'erreur du modèle. Pour la classification, on utilise la **Cross-Entropy Loss** :

$L = - \Sigma\, y_i \times \log(p_i)$

où y_i est la vraie classe (0 ou 1) et p_i est la probabilité prédite.

Interprétation : Plus la loss est basse, mieux le modèle prédit. Une loss de 0 serait parfaite. Au début du fine-tuning, la loss est ~1.4 (équivalent random sur 4 classes). Un bon fine-tuning amène la loss vers 0.3-0.5.

F1 Macro

La F1 combine Précision et Rappel en une seule métrique :

Précision = TP / (TP + FP) → "quand je dis Classe X, j'ai raison combien de fois ?"

Rappel = TP / (TP + FN) → "parmi tous les vrais X, combien j'en trouve ?"

F1 = 2 × (P × R) / (P + R)

F1 Macro = moyenne de F1 sur les 4 classes

Pourquoi F1 Macro et pas Accuracy ? Parce que F1 Macro traite chaque classe également. Si ton modèle ignorait complètement une classe, l'accuracy resterait haute mais la F1 Macro chuterait.

Échelle : 0 = nul, 1 = parfait. Pour AG News, un bon modèle atteint 0.90+.

Sharpness (Netteteté du minimum)

Concept fondamental de Keskar et al. (2017) : tous les minimums de la loss ne se valent pas.

Minimum plat (flat minimum) :

```
      ___________
     /           \
------/            \------
```

→ Bon : la loss reste basse même si les poids bougent légèrement

→ Généralise bien sur de nouvelles données

Minimum pointu (sharp minimum) :

```
          |
         /|\
---------/ | \---------
```

→ Mauvais : la loss explose si les poids bougent un peu

→ Mémorise les données d'entraînement, généralise mal

Formule de sharpness dans notre code :

$S = (1/N) \Sigma |L(\theta + \alpha d_i) - L(\theta)|$

On perturbe les poids dans une direction aléatoire et on mesure combien la loss change. Plus S est petit, plus le minimum est plat, plus le modèle généralisera bien.

Random Search (Recherche Aléatoire d'Hyperparamètres)

Au lieu de tester toutes les combinaisons (Grid Search = très coûteux) ou de faire confiance à son intuition (manual search), on tire aléatoirement des configurations dans un espace défini :

Hyperparamètres cherchés :

- Learning Rate : entre 1e-6 et 5e-4 (échelle log)
- Batch Size : {8, 16, 32}
- Warmup Steps : entre 0 et 200
- Num Epochs : entre 1 et 4

Pour chaque trial :

1. Tire une config aléatoire
2. Fine-tune le modèle avec cette config
3. Évalue sur val set
4. Sauvegarde les résultats

Pourquoi ça marche ? Bergstra & Bengio (2012) ont montré que le Random Search est souvent plus efficace que Grid Search car tous les hyperparamètres n'ont pas la même importance — le LR est crucial, le batch size l'est moins. Le Random Search explore mieux les hyperparamètres importants.

Warmup (Échauffement du Learning Rate)

Sans warmup :

LR = 2e-5 dès le début

→ Risque de détruire les poids pré-entraînés au 1er pas

Avec warmup (ex: 100 steps) :

Steps 0-100 : LR monte de 0 à 2e-5 progressivement

Steps 100+ : LR redescend linéairement

Pourquoi ? Les poids pré-entraînés de DistilBERT représentent des années de connaissance linguistique. Un LR trop élevé au départ les "casse" avant même que la tête de classification ait appris quoi que ce soit.

---

4. Comment Interpréter Chaque Figure

Figure 00 — Distribution des Classes

Ce que tu vois : 4 barres égales (30,000 articles chacune pour le train complet).

Ce que ça signifie : AG News est un dataset équilibré. C'est une bonne nouvelle — ton modèle n'aura pas de biais vers une classe plus fréquente.

Implication : L'Accuracy et la F1 Macro seront proches (sur un dataset déséquilibré, elles divergeraient beaucoup).

Figure 1 — Courbes de Convergence

Panneau gauche (Train Loss) :

Bonne convergence : Surapprentissage : Non-convergence :

```
\                 \___            ~~~~~~~~~~
 \___                            ↗
    ___                         ↗
```

- La courbe doit descendre régulièrement puis se stabiliser
- Des oscillations indiquent un LR trop élevé
- Une descente très lente indique un LR trop faible
- Le trait lisse est la moyenne mobile (rolling average sur 5 steps) — ignore les pics

Panneau droit (Val F1) :

- Doit monter puis se stabiliser (early stopping si ça redescend)
- Si Val F1 monte mais Train Loss continue de descendre sans limite → surapprentissage
- Le meilleur run par optimiseur est mis en avant (trait épais)

Ce qu'on cherche : AdamW converge généralement plus vite et plus stable. SGD peut être chaotique mais peut atteindre de meilleures valeurs finales. Adafactor est quelque part entre les deux.

Figure 1b — Tous les Trials

Ce que tu vois : Toutes les courbes Val F1 pour chaque optimiseur, trait épais = meilleur run.

Comment lire :

- **Dispersion faible** (courbes groupées) → l'optimiseur est robuste au choix du LR
- **Dispersion forte** (courbes éparpillées) → très sensible au LR, risqué en production
- **Courbes plates** → le LR était mauvais, le modèle n'a pas appris

Implication pratique : Un optimiseur robuste est préférable même s'il n'atteint pas le maximum absolu — on ne peut pas toujours passer des heures à tuner le LR.

Figure 2 — Heatmap Optimiseur × LR

Ce que tu vois : Un tableau coloré. Lignes = optimiseurs, colonnes = plages de LR, couleur = meilleure F1 atteinte dans cette case.

Comment lire :

- **Case rouge foncé / bleu foncé** = bonne combinaison optimiseur + LR
- **Case pâle / blanche** = mauvaise combinaison ou pas de trial dans cette case

Questions à se poser :

1. Y a-t-il une colonne (plage LR) qui est bonne pour tous les optimiseurs ? → LR universel
2. Y a-t-il un optimiseur qui est bon sur plusieurs colonnes ? → robuste au LR

3. Y a-t-il un optimiseur qui n'a de bonne case que sur une seule colonne ? → très sensible

---

Figure 2b — Scatter LR vs F1

Ce que tu vois : Chaque point = 1 trial. Axe X = LR (log), Axe Y = F1 val. Étoile = meilleur run par optimiseur.

Patterns à chercher :

- **Pic au centre** → il y a un LR optimal dans une plage précise
- **Plateau** → l'optimiseur est robuste sur une large plage de LR
- **Chute à gauche** → LR trop faible, le modèle n'apprend pas
- **Chute à droite** → LR trop élevé, le modèle diverge

Figure 3 — Loss Landscape 1D

Ce que tu vois : Pour chaque optimiseur, on perturbe ses meilleurs poids dans une direction aléatoire et on mesure comment la loss varie. L'axe X est l'intensité de perturbation (α=0 = poids originaux).

Comment lire :

```
Courbe plate :          Courbe pointue :

    ___________                 |
   /         \              /|\
----/          \----        / | \
  -ε   0   +ε            -ε  0  +ε


Sharpness faible → bon     Sharpness élevée → risqué
```

- La courbe la **plus plate** indique le minimum le mieux généralisable
- La courbe la **plus basse** (loss minimale) indique le meilleur entraînement absolu
- Ces deux propriétés peuvent ne pas correspondre au même optimiseur — c'est la tension principale de P01

Figure 4 — Analyse Sharpness

Panneau gauche (barplot Sharpness) :

- Plus la barre est **courte** = minimum plus plat = **meilleure généralisation**
- Comparer directement les 3 optimiseurs

Panneau droit (scatter Sharpness vs F1) :

- **Coin haut gauche** = idéal (F1 haute + Sharpness faible)
- **Coin bas droit** = pire (F1 basse + Sharpness élevée)
- Les diamants (♦) sont les F1 moyens par optimiseur

Question centrale : L'optimiseur qui a la meilleure F1 a-t-il aussi la sharpness la plus faible ? Si oui, il domine clairement. Si non, il y a un compromis à discuter dans le rapport.

Figure 5 — Tableau Récapitulatif

Colonne	Ce que ça mesure	Bon signe
Best F1	Meilleure F1 atteinte sur l'ensemble de validation	Élevé
Mean F1	F1 moyenne sur tous les trials	Élevé + proche de Best F1
Best Acc	Meilleure accuracy	Élevé
Temps moy	Temps moyen par trial	Faible
N Trials	Nombre de trials réussis	Proche du prévu
Sharpness	Netteté du minimum	Faible

Comment lire le gradient de couleur :

- Vert foncé sur Best F1 = meilleur
- Rouge clair sur Sharpness = meilleur (sharpness la plus faible)

Figure 6 — Distributions (Boxplots)

Ce que tu vois : La distribution statistique des performances sur tous les trials.

Anatomie d'un boxplot :

```
   ○ ← outlier (valeur aberrante)
   |
 -------  ← Q3 (75ème percentile)
 |     |
 |  –  |  ← médiane (Q2)
 |     |
 -------  ← Q1 (25ème percentile)
   |
   |  ← min (sans outliers)
```

Ce qu'on cherche :

- **Boîte haute** = bonnes performances médianes
- **Boîte étroite** = résultats consistants (peu de variance)
- **Boîte large** = résultats très variables selon le LR tiré
- **Outliers** = quelques très bons ou très mauvais runs

Idéal : Boîte haute ET étroite = bon ET robuste.

---

5. Comment Rédiger la Conclusion du Rapport

Après avoir tout analysé, tu dois répondre à cette question : **lequel des 3 optimiseurs recommandes-tu pour fine-tuner DistilBERT sur une tâche de classification de texte, et pourquoi ?**

Voici le cadre d'analyse :

Critère 1 — Performance brute : Qui a la meilleure Best F1 ? (Figure 5, Figure 2)

Critère 2 — Robustesse : Qui a la plus petite variance ? (Figure 6, Figure 1b) Un optimiseur qui ne performe bien que sur un LR précis est risqué en pratique.

Critère 3 — Généralisation : Qui a la sharpness la plus faible ? (Figure 3, Figure 4) Un minimum plat généralise mieux sur de nouvelles données.

Critère 4 — Efficacité : Qui est le plus rapide ? (Figure 5, colonne Temps) Important si les ressources sont limitées.

La tension classique que tu trouveras probablement : **AdamW aura la meilleure F1 brute mais une sharpness plus élevée, SGD aura une sharpness plus faible mais sera moins robuste au LR, Adafactor sera le plus économe en mémoire mais légèrement en dessous en performance.** Ta discussion doit articuler ce compromis selon le contexte d'utilisation (production, recherche, ressources limitées).